

Building a Simple AI Agent, Part 2

Once the Agent has generated a response, we need to interface the agent with the environment. This involves figuring out how the Agent's response corresponds to an action in the environment. Once the correct action is determined, the interface can execute the action and later provide the Agent feedback on the result of the action.

Step 3: Parse the Response

After generating a response, the next step is to extract the intended action and its parameters from the LLM's output. The response is expected to follow a predefined structure, such as a JSON format encapsulated within a markdown code block. This structure ensures the action can be parsed and executed without ambiguity.

In the code, this is accomplished by locating and extracting the content between the ````action` markers. If the response does not include a valid action block, the agent defaults to a termination action, returning the raw response as the message.

```
def parse_action(response: str) -> dict:
    """Parse the LLM response into a structured action dictionary."""
    try:
        response = extract_markdown_block(response, "action")
        response_json = json.loads(response)
        if "tool_name" in response_json and "args" in response_json:
            return response_json
        else:
            return {"tool_name": "error", "args": {"message": "You must respond with a JSON tool invocation."}}
    except json.JSONDecodeError:
        return {"tool_name": "error", "args": {"message": "Invalid JSON response. You must respond with a JSON tool invocation."}}
```

This parsing step is critical to ensuring the response is actionable. It provides a structured output, such as:

```
{
  "tool_name": "list_files",
  "args": {}
}
```

By breaking down the LLM's output into `tool_name` and `args`, the agent can precisely determine the next action and its inputs.

If the LLM response does not contain a valid action block, the agent defaults to an error message, prompting the LLM to provide a valid JSON tool invocation. The error message appears to have come from the "user". This fallback mechanism ensures the agent can recover if it starts outputting invalid responses that aren't in the desired format.

Step 4: Execute the Action

Once the response is parsed, the agent uses the extracted `tool_name` and `args` to execute the corresponding function. Each predefined tool in the system instructions corresponds to a specific function in the code, enabling the agent to interact with its environment.

The execution logic involves mapping the `tool_name` to the appropriate function and passing the provided arguments:

```
@action("list_files") -> "list_files"
def list_files():
    result = (result["list_files"])
    return result["list_files"]

@action("read_file") -> "read_file"
def read_file(file_name):
    result = (result["read_file(action['args']['file_name'])"])
    return result["read_file(action['args']['file_name'])"]

@action("print_message") -> "print_message"
def print_message(message):
    result = (result["print_message"])
    return result["print_message"]
```

For example, if the action specifies `list_files` with empty `args`, the `list_files()` function is called, and the agent returns the list of files in the directory. Similarly, a `read_file` action extracts the filename from the arguments and retrieves its content.

The execution step is the point where the agent performs tangible work, such as interacting with files or printing messages to the console. It bridges the decision-making process with concrete results that feed back into the agent's memory for subsequent iterations.